AAR: AI-Based System Call Anomaly Detection & Response

Layer 3 (Model Scoring) and Layer 4 (Evaluation & Benchmarking)
From the first version (v1) to the improved version (v2)

Akshat Kapur (24BDS0318) · Aahav Yadav (24BKT0138) · Rishab Jajodia (24BDS0268)
Repository: github.com/AkshatKapur1/AAR-System-Call-Anomaly-Detection · October 2026

Summary

Three anomaly detectors - a Hidden Markov Model (HMM), a GRU and an LSTM - learn what **normal** system-call behaviour looks like from the ADFA-LD, LID-DS-2021 and PLAID datasets, and flag recordings that deviate from it. Layer 3 turns each trained model into an anomaly score per recording; Layer 4 sets thresholds, measures detection and picks the best model for the response layer (Layer 5).

The first version (v1) separated attacks from normal recordings only weakly. We identified seven causes, retrained the models, added context-aware scoring and per-environment thresholds, and replaced the evaluation with a protocol in which no choice is made on the data used to report results.

Best model: LSTM (final 70% of test: 583 normal, 716 attacks)	**v1**	**v2**
ROC-AUC (separation of attacks from normal; 0.5 = guessing)	0.676	0.835
Heartbleed (LID-DS CVE-2014-0160) ROC-AUC	0.441	0.871
Strict threshold (~1% false alarms): attacks caught	107 / 716	107 / 716
Strict threshold: normal recordings wrongly flagged	8 / 583	6 / 583
Balanced alert threshold: attacks caught	-	638 / 716 (89.1%)
Balanced alert threshold: normal correctly passed	-	381 / 583 (65.4%)
Balanced alert threshold: F1	-	0.820

v1 values are the original models re-scored on exactly the same 70% of test records, so the comparison is like-for-like. The Heartbleed v1 value is the original LSTM's AUC on that CVE over the full test set.

1. Data: the train, validation and test files

Layers 1-2 (parsing, de-duplication, balancing and splitting) produced three files on GitHub. Each recording is a sequence of system-call names, converted to integer IDs with a 222-entry vocabulary (`split/vocab.json`, built from train only; unseen calls become <UNK>).

File	Recordings	Normal	Attack	7-syscall windows
split/train.jsonl.gz	3888	3888	0	8,771,179
split/val.jsonl.gz	833	833	0	1,719,403
split/test.jsonl.gz	1860	834	1026	3,370,804

Attacks appear **only in test** (semi-supervised design): the models learn normal behaviour only, so they can flag attacks they have never seen. Test attacks: ADFA-LD 740 (Adduser, Hydra FTP/SSH, Meterpreter, Java Meterpreter, Web Shell), LID-DS 116 (CVE-2014-0160 Heartbleed, CVE-2012-2122 MySQL auth bypass) and PLAID 170 (cowroot, ftp, nginx, privesc, redis, ssh).

How each file was used

File	Used for	Never used for
train	Training all three models (v1: a 2-6% sample of its 7-syscall windows; v2: all of it, ADFA-LD weighted x3). Scored afterwards only to check for overfitting.	Thresholds, results
val (normal only)	All thresholds (95th / 99th / 99.5th percentile of normal scores, global or per dataset) and score normalisation. v2: early stopping for GRU/LSTM, choosing the number of HMM states.	Measuring attack detection (it has no attacks)
test - TUNE part (30%, 559 records)	v2 only: every choice - scoring mode, aggregation, threshold type, best model, balanced alert threshold.	Training, reported results
test - FINAL part (70%, 1,301 records)	v2 only: the reported results, computed once. The v1 models were re-scored on the same records for comparison.	Any choice or tuning

The TUNE/FINAL partition (`split/test_partition.json`) is stratified by source, label and attack type with a fixed seed (42), so every dataset and attack type appears in both parts. `test.jsonl.gz` itself is unchanged. In v1 there was no TUNE part: the whole test set was used both to report results and to pick the best model - one of the problems fixed in v2.

2. Version 1

2.1 Models (trained by the team before Layer 3 scoring)

	HMM (Track A)	GRU (Track B)	LSTM (Track C)
Input	7-gram windows from train_windows_n7.jsonl	same	same
Sample used	200,000 of 8.77M windows (2.3%)	500,000 (5.7%)	500,000 (5.7%)
Size	8 hidden states	embed 32, 1 x 64 GRU	embed 32, 1 x 64 LSTM
Training	Baum-Welch, 20 iterations	5 epochs, Adam	5 epochs, Adam
Train time	430.7 s	84.0 s	47.2 s
Normal-data fit	-	final loss 0.658	final loss 0.663

2.2 Layer 3 v1: turning the models into anomaly scores

- **Windows.** Every val/test recording was cut into overlapping 7-syscall windows, sliding one call at a time and never crossing recordings - the same method used for training (checked: applying it to train reproduces all 8,771,179 training windows).

- **Window score** (higher = more anomalous). HMM: -log $P(s_1 ... s_7)$ via the forward algorithm. GRU/LSTM: mean over t = 1..6 of -log $P(s_{t+1} \mid s_1 ... s_t)$.
- **Recording score** = mean of the 5% most anomalous windows (fixed in advance).
- **Verification.** Rescoring the held-out windows saved at training time reproduced the stored values (max difference 5e-7), proving the models were loaded and scored exactly as trained.
- **Overfitting check.** Median normal scores were almost identical for train, val and test (LSTM 3.24 / 3.27 / 3.26), so the models did not overfit.

2.3 Layer 4 v1: first evaluation (full test set)

Thresholds were the 95th / 99th / 99.5th percentile of each model's normal val scores - one global threshold per model. The 99th percentile (~1% false alarms) was the primary operating point.

Model	ROC-AUC	PR-AUC	Attacks caught @p99	False alarms @p99	Precision	F1
HMM (Track A)	0.527	0.629	72 / 1022	6 / 834	92.3%	0.131
GRU (Track B)	0.672	0.736	118 / 1022	13 / 834	90.1%	0.205
LSTM (Track C)	0.665	0.733	144 / 1022	14 / 834	91.1%	0.244

Model	ADFA-LD	LID-DS	PLAID	CVE-2014-0160 (Heartbleed)
HMM (Track A)	0.432	0.625	0.815	0.311
GRU (Track B)	0.594	0.592	0.899	0.460
LSTM (Track C)	0.587	0.523	0.900	0.441

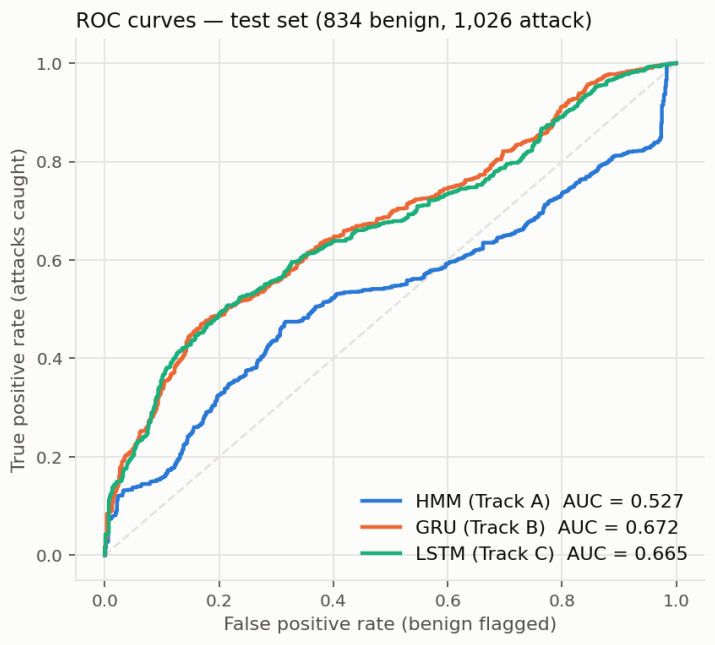


Figure 1 - v1 ROC curves on the full test set. The HMM is close to the diagonal (random guessing).

2.4 Problems found in v1

#	Problem	Evidence
1	Too little training data	Models saw only 2-6% of the 8.77M training windows.
2	ADFA-LD under-represented	14% of training windows (LID-DS 44%, PLAID 42%); the capping sample_weight of 3.04 for ADFA-LD was ignored. ADFA-LD holds 740 of 1,026 attacks and had the worst AUC (0.43-0.59).
3	Very short memory	Each window was judged from its own 7 calls only; the HMM never learned how one window flows into the next.

#	Problem	Evidence
4	Small, briefly trained models	1 layer x 64 units, 5 epochs; HMM with 8 states.
5	One threshold for all environments	Global p99 set by the noisiest normal data (PLAID p99 7.4 vs LID-DS 4.3 for the LSTM), so LID-DS/ADFA attacks had to clear a bar far above their own normal range.
6	Aggregation fixed before testing	Averaging all windows gave AUC 0.76 vs 0.67 for the chosen top-5% rule, but switching after seeing test would bias results.
7	Best model chosen on test	The same data was used to choose and to report.

3. Version 2: changes made

Problem	Change in v2	Where
1 Too little data	Train on **all** of split/train (8.79M calls)	models/*
2 ADFA-LD under-represented	Use sample_weight: weighted loss (GRU/LSTM), ADFA-LD records repeated x3 (HMM)	models/*
3 Short memory	GRU/LSTM learn from 128-call chunks; HMM from whole recordings. New **stream** scoring: each 7-call window scored with the whole preceding recording as context	models/*, codes/scoring
4 Small models	GRU/LSTM: embed 64, 2 x 128, dropout 0.1, up to 12 epochs, early stopping on val loss. HMM: 8/16/32 states, chosen on val log-likelihood	models/*
5 One threshold	Per-dataset thresholds and normalisation z = (s - median) / (p99 - median) from normal val	codes/evaluation
6 Fixed aggregation	Five aggregations (mean, max, top 1/5/10%) chosen on TUNE	codes/scoring, evaluation
7 Choosing on test	Test split into TUNE (30%) / FINAL (70%); all choices on TUNE, results on FINAL only	codes/split, evaluation
Strict-only threshold	Second, **balanced alert** threshold (max Youden's J on TUNE)	codes/evaluation

3.1 Retrained models

	HMM (Track A)	GRU (Track B)	LSTM (Track C)
Training data	All train recordings, full sequences, ADFA-LD x3	All train, 128-call chunks, weighted	same as GRU
Size	32 states (8/16/32 tried)	216,414 parameters	274,270 parameters
Normal val fit	log-lik/call -2.47 (8) -> -1.85 (16) -> -1.55 (32)	val loss 0.429 (v1 ~0.65)	val loss 0.424 (v1 ~0.65)
Epochs / iterations	<= 25 EM iterations	12	12
Train time	1,722 s	2,285 s	1,605 s

GRU and LSTM share one code file (models/sequence_model_common.py) and differ only in the recurrent cell, which keeps the comparison fair. No attack data is used anywhere in training.

3.2 Layer 3 v2: scoring

- Windows are still **7 system calls (n = 7)**, as in the architecture. Each window is scored in two modes: **window7** (the model sees only the window, as in v1) and **stream** (the model reads the recording from the start and scores the window's calls 2-7 given everything before them).
- Window scores are aggregated per recording five ways: mean, max, and mean of the top 1% / 5% / 10%.
- **Check:** a recording's first window has no earlier context, so both modes must agree on it (HMM: window7 = -log $P(s_1)$ + 6 x stream). Max difference: HMM 1.4e-14, GRU 6.2e-07, LSTM 5.6e-07.
- Outputs: scoring/{hmm,gru,lstm}_{train,val,test}_record_scores.csv with 10 score columns each.

3.3 Layer 4 v2: fair evaluation protocol

- **Thresholds** from normal val only: 95th / 99th / 99.5th percentile, global or per dataset.
- **20 candidates per model** (2 modes x 5 aggregations x 2 threshold types) scored on TUNE. Rule fixed in advance: highest TUNE ROC-AUC; within 0.01, higher TUNE recall at p99. Best model chosen the same way.
- **Two operating points: kill** = p99 of normal val (~1% false alarms); **balanced alert** = the cutoff maximising (% attacks caught + % normal passed) on TUNE.
- **Reporting:** the selected configurations evaluated once on FINAL, next to the v1 baseline on the same records.

Model	Selected mode	Aggregation	Threshold type	TUNE ROC-AUC
HMM (Track A)	window7	mean	per-dataset	0.645
GRU (Track B)	stream	mean	per-dataset	0.835
LSTM (Track C)	stream	mean	per-dataset	0.847

Best model on TUNE: **LSTM**. What mattered most (TUNE): stream scoring beat window7 for GRU/LSTM by about 0.06-0.07 AUC; averaging all windows beat max/top-k; per-dataset thresholds caught more attacks at equal or fewer false alarms (LSTM 14.7% vs 13.3%, GRU 18.6% vs 15.0%).

4. Results: v1 vs v2 (FINAL 70% of test: 583 normal, 716 attacks)

Model	ROC-AUC v1 -> v2	PR-AUC v2	Caught @p99 v1 -> v2	False alarms @p99 v1 -> v2	Precision @p99 v2
HMM (Track A)	0.539 -> **0.683**	0.728	51 -> **88**	5 -> **9**	90.7%
GRU (Track B)	0.682 -> **0.821**	0.848	83 -> **110**	8 -> **4**	96.5%
LSTM (Track C)	0.676 -> **0.835**	0.856	107 -> **107**	8 -> **6**	94.7%

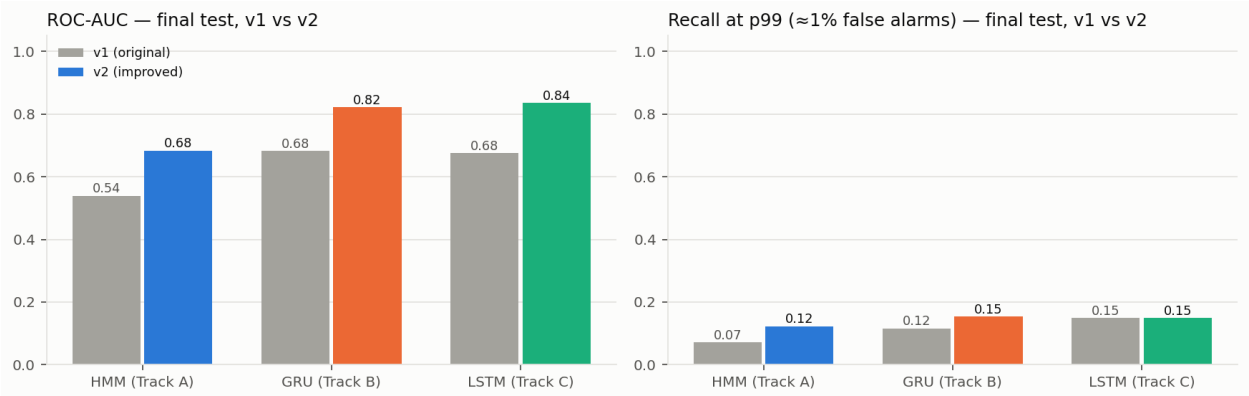


Figure 2 - ROC-AUC and recall at the strict threshold, v1 vs v2, on the same FINAL records.

4.1 Balanced alert threshold

Model	Attacks detected	Normal correctly passed	False alarms	Precision	F1	Balanced acc.
HMM (Track A)	299 / 716 (41.8%)	483 / 583 (82.8%)	100	74.9%	0.536	0.623
GRU (Track B)	624 / 716 (87.2%)	381 / 583 (65.4%)	202	75.5%	0.809	0.763
LSTM (Track C)	638 / 716 (89.1%)	381 / 583 (65.4%)	202	76.0%	0.820	0.772

The LSTM gets 1019 of 1,299 recordings right at the balanced threshold. It catches ~6x more attacks than at the strict threshold, at the cost of about 1 in 3 normal recordings being flagged - suitable for alerting and review, not for killing processes.

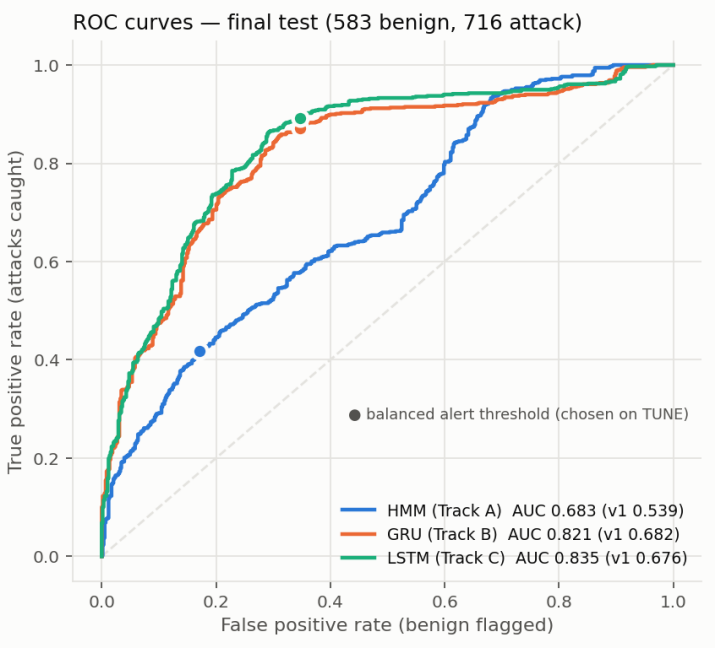


Figure 3 - v2 ROC curves on FINAL; dots mark each model's balanced alert threshold (chosen on TUNE).

4.2 Per dataset (LSTM)

Dataset	ROC-AUC v1 -> v2	Attacks caught (balanced)	Normal passed (balanced)	Attacks caught (strict)
ADFA-LD	0.599 -> **0.789**	464 / 518	192 / 328	50 / 518
LID-DS	0.530 -> **0.829**	60 / 81	89 / 127	24 / 81
PLAID	0.911 -> **0.950**	114 / 117	100 / 128	33 / 117
LID-DS/CVE-2014-0160	0.441* -> **0.871**	56 / 77	53 / 65	20 / 77
LID-DS/CVE-2012-2122	1.000* -> **1.000**	4 / 4	36 / 62	4 / 4

* v1 per-CVE values are from the full test set (the v1 baseline stored per dataset only).

4.3 Per attack type (LSTM, share of attacks caught)

Dataset	Attack type	n	Strict (p99)	Balanced alert	AUC vs same-dataset normal
ADFA-LD	Adduser	64	1.6%	78.1%	0.707
ADFA-LD	Hydra_FTP	113	17.7%	92.0%	0.840
ADFA-LD	Hydra_SSH	122	13.9%	93.4%	0.843
ADFA-LD	Java_Meterpreter	85	0.0%	92.9%	0.745
ADFA-LD	Meterpreter	51	0.0%	92.2%	0.756
ADFA-LD	Web_Shell	83	14.5%	84.3%	0.770
LID-DS	CVE-2012-2122	4 (small)	100.0%	100.0%	1.000
LID-DS	CVE-2014-0160	77	26.0%	72.7%	0.820
PLAID	cowroot	11 (small)	0.0%	100.0%	0.955
PLAID	ftp	13 (small)	0.0%	92.3%	0.925
PLAID	nginx	19 (small)	89.5%	100.0%	0.991
PLAID	privesc	4 (small)	25.0%	100.0%	0.973
PLAID	redis	48	31.2%	100.0%	0.961
PLAID	ssh	22	0.0%	90.9%	0.897

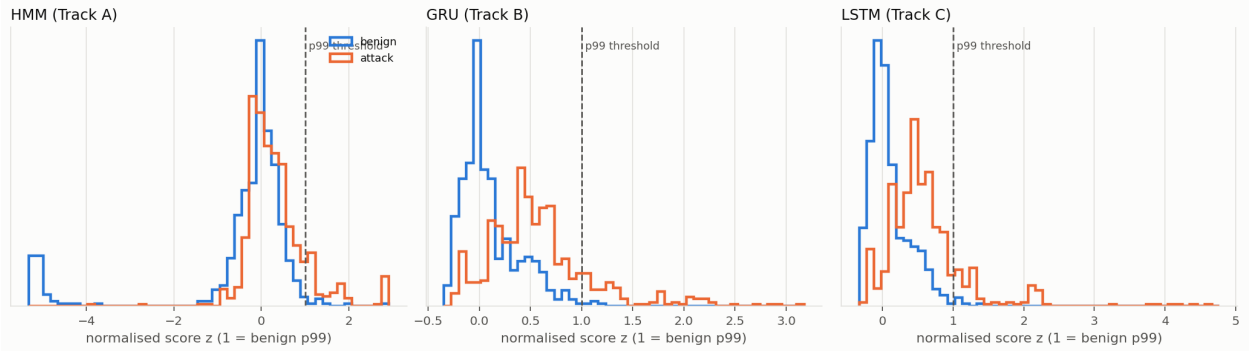


Figure 4 - v2 normalised scores on FINAL (0 = typical normal, 1 = normal p99 = strict threshold).

4.4 Cost

Model	Train time (v1 -> v2)	Inference (selected mode)	Parameters
HMM (Track A)	430.7 s -> 1,722 s	68.0 ms / 1,000 windows	8,160
GRU (Track B)	84.0 s -> 2,285 s	19.4 ms / 1,000 windows	216,414
LSTM (Track C)	47.2 s -> 1,605 s	8.3 ms / 1,000 windows	274,270

Peak memory while scoring all splits: 6.0 GB (mostly holding the 8.77M train windows). The LSTM is the fastest scorer and the most accurate.

5. Why not every attack is caught

- Catching **all** attacks would flag 97% of normal recordings (LSTM FPR at TPR = 1): the least suspicious attacks score below almost every normal recording.
- Most attacks score between typical normal (z = 0) and the strangest 1% of normal (z = 1), e.g. ADFA Adduser / Meterpreter median z ~0.43, PLAID ssh 0.20 - suspicious, but below a strict line.
- Many attacks are made of ordinary system calls; what makes them malicious is the arguments (which file, user or address), which the name-only data does not contain. Attacks that use calls never seen in training are caught 83% of the time at the strict threshold vs 13% for the rest.
- Attack recordings also contain normal activity (e.g. LID-DS records everything after the exploit starts), which dilutes a per-recording score.

6. Limitations and next step

- Name-only features: no arguments, return values or timing (needs the raw datasets, not on GitHub).
- GRU/LSTM learn from 128-call chunks; detection is still per 7-call window, but training no longer uses the 7-gram file directly - a deliberate deviation from the architecture diagram.
- FINAL is 70% of test; small attack groups (CVE-2012-2122 n=4, privesc n=4, cowroot n=11, ftp n=13, nginx n=19) are indicative only.
- At the strict threshold the LSTM's attack recall did not rise (GRU 83 -> 110, HMM 51 -> 88) - a few unusual normal recordings set that bar.

Layer 5 (response) uses the best model's stream scores over time: a per-process confidence that rises with anomalous windows and decays otherwise, the balanced threshold to raise alerts, and hysteresis plus the strict threshold to trigger the kill switch - catching attacks early while keeping false kills rare.

Appendix: files and commits on GitHub

Path	Contents
split/test_partition.json	TUNE / FINAL record indices of test
models/HMM, models/gru, models/lstm	v2 models, training scripts and training_report.json
models/sequence_model_common.py	shared GRU/LSTM model, training and scoring code
codes/scoring/ + scoring/	Layer 3 script, per-recording scores for train/val/test, scoring_report.json
codes/evaluation/ + evaluation/	Layer 4 script, thresholds, metrics, plots, baseline, best_model.json

Commit	Change
2aa4328, 5c1f5ea	Layer 3 v1: val/test scoring, then train scores
a1f3789	Layer 4 v1: first evaluation
a9c6395	v2: retrained models, stream scoring, TUNE/FINAL protocol
e90a3f2	Balanced alert threshold